ChatGPT

Robust CV Pipeline Plan for Qwirkle Connect Score Calculator

Board ROI Calibration and Grid Mapping

Step 1: Extract the board Region of Interest (ROI) and align it to a grid. We use the provided **empty board image** as a reference to locate and warp the board in all game frames. A fixed camera is used, but slight shifts or rotations can occur between frames. To handle this, we perform a **perspective calibration**:

- **Find Board Corners:** Detect the four corners of the 16×16 board in the empty image (e.g. via edge detection or Hough lines for the outer grid boundary). Alternatively, use feature matching between the empty board and the first frame (`j00.jpg`) to find the board region. This yields the pixel coordinates of the board's corners in the image [1].
- **Apply Perspective Transform:** Compute a homography to map these corner points to a fixed 16×16 grid coordinate system. For example, map corners to (0,0), (16*cell_size,0), (0,16*cell_size), (16*cell_size, 16*cell_size). Then warp each game image to a top-down view. This corrects small rotations/perspective so each cell aligns with a consistent coordinate grid.
- **Define Grid Cell Coordinates:** After warping, each of the 256 board cells corresponds to a known pixel region. We assign row numbers 1–16 (for the vertical axis) and columns A–P (horizontal). For example, the top-left cell is `1A` and bottom-right is `16P`. We record the pixel bounding box for each cell (based on the cell size from calibration).
- **Handle Misalignment:** If the camera slightly shifts in later frames, we can adjust the ROI by **aligning features** of the new frame to the calibrated reference. For instance, compute a translational offset by cross-correlating the current frame with the empty board (focusing on the grid lines or known tile positions) and apply a small shift to recenter. This ensures the cells map consistently across frames. We leave a small margin when cropping to avoid cutting off tiles if alignment is a few pixels off.
- **Verify Alignment:** Using the initial anchor tiles (see next step) as reference points – their colored shapes should fall roughly in the center of their assigned cells. If we notice a systematic offset (e.g. all anchors appear shifted down-right within cells), we tweak the homography or ROI offset accordingly. This prevents cases where shapes touching a cell border get cut off due to a cropping error. By the end of this step, we have a function `image_to_board(coords)` that converts image pixel coordinates to board row/col (and vice versa), allowing precise mapping of tile locations.

Initial Anchor Detection on the Starting Board (j00)

Step 2: Identify the 24 anchor tiles on the initial board and initialize board state. The first image `j00.jpg` shows the board with **24 randomly placed anchor tiles on the black bonus squares** [2]. We must detect these anchors and record their positions and identities:

- **Detect Occupied Cells:** Compare `j00.jpg` to the empty board. We perform a **cell-wise image differencing**: for each cell region, compute the difference between `j00` and the empty board

image (e.g. sum of absolute pixel differences or structural similarity). Cells with significant difference contain a tile. This yields ~24 occupied cells. We expect exactly 24 due to game rules [2]; if we get a different number, adjust the threshold or check alignment.

- **Segment Each Anchor Tile:** For each detected occupied cell, isolate the tile's symbol. The physical Qwirkle tiles have black backgrounds with a colored shape. We crop the cell and apply color thresholding to get the symbol:
- Convert to HSV and mask out low-value pixels (the black tile) – e.g. require **Value > T1** and **Saturation > T2** to keep only the bright colored regions. This yields a binary mask of the symbol. We may also use edge detection (Canny) to outline the shape if color segmentation is weak (especially for dark colors like purple/blue).
- Apply morphological closing if needed so the symbol is one continuous contour (some shapes might have narrow gaps).
- **Contour Extraction:** Find the largest contour in the mask – this should correspond to the symbol's outline. Simplify or smooth it slightly (e.g. approxPolyDP for less jagged edges), but preserve shape fidelity.
- **Classify Anchor Shape & Color:** Use the template-based methods (prepared in the next section) to identify each anchor tile's shape (1–6) and color (R, B, G, Y, O, W). This gives an initial board mapping of coordinates to tile type (e.g. `12D = 3G` meaning shape3–Green at 12D). We store these in a data structure `board_state[(row,col)] = (shape_id, color_id)`.
- **Initialize Board State:** After this step, `board_state` contains 24 entries for the anchors. This state will be updated as new tiles are placed each move. It will be used for validating moves and scoring.

Early validation: We can cross-check that anchor tiles are on the expected **black bonus squares** (given by game design). This indirectly validates that our ROI mapping is correct (anchors should coincide with known special square coordinates if orientation is right, next step). If any anchor seems to lie on a non-bonus cell (per the standard board layout), it flags a possible orientation or detection error, which we'll address below.

Determining Board Orientation and Bonus Squares

Step 3: Determine the orientation of the board quadrants and map out the +1/+2 bonus cells. The Qwirkle Connect board is comprised of four 8×8 quadrants that can be rotated in various configurations [1]. The **black bonus squares** (where the 24 anchors sit) have printed **1 or 2 point values** on them, divided equally among the quadrants (likely 12 one-point and 12 two-point spaces in total). We need to figure out which bonus value each anchor cell corresponds to.

- **Known Quadrant Template:** From the game rules, we know the layout of bonus squares in a single quadrant (the recommended setup). For example, the quadrant might have black bonus spaces at specific relative positions (e.g. a certain pattern or symmetric arrangement). We create a reference list of coordinates for bonus squares in one quadrant in its default orientation (e.g. (rows, cols) within that 8×8 quadrant that are black). We also know which of those are 1-point and which are 2-point (from the game manual).
- **Match Anchors to Quadrants:** Divide the detected anchor coordinates into four groups corresponding to the four quadrants of the board (top-left, top-right, bottom-left, bottom-right regions). Each group of ~6 anchors should match the quadrant's bonus pattern under some rotation. For each quadrant group, try all 4 rotations of the reference pattern (0°, 90°, 180°, 270°) and see which rotation aligns with the anchor positions in that region. The correct rotation will have a high match (ideally all 6 anchors coincide with the rotated pattern's coordinates).

- **Resolve Orientation:** Assign the determined orientation to each quadrant. Now we know exactly which board coordinates are 1-point bonus squares and which are 2-point. (For example, after orientation fix, we might determine that cells `8H, 8I, 9H, 9I` are +2, etc. – depending on quadrant configuration.)
- **Store Bonus Mapping:** Create a set or lookup structure for bonus cells: e.g. `bonus_map[(r,c)] = 1/2` if that cell is a bonus, or 0 if normal. This will be used in score calculation.
- **Verification:** Ensure all 24 anchors fall on cells marked as bonus (they should by rule). If any anchor lies outside the derived bonus map, we likely picked a wrong orientation – re-evaluate the matching or consider if quadrant pieces might have been swapped (if applicable). In a robust solution, you might attempt alternative quadrant assignments if one approach doesn't yield a perfect match, but typically the recommended setup will match uniquely. Once this is consistent, our board coordinate system is fully understood (including special squares).

Mitigation note: This orientation step is crucial for scoring the +1/+2 correctly. If uncertain, we can double-check against known scoring later. For instance, if a move placing a tile on what we think is a +2 yields a score mismatch (off by 1) compared to expected, that could hint the cell was actually +1 and our orientation was off. However, by using all anchor positions for calibration, we minimize this risk upfront.

Template Extraction for Shape and Color Recognition

Step 4: Prepare reference templates for all 6 shapes and 6 colors using the provided auxiliary image `imagini_auxiliare/all_tiles.jpg` (the board with all 36 shape–color combinations placed cleanly). We do **not** use any game frames to derive templates (to avoid bias); instead we rely on this controlled reference. The template preparation involves:

- **Isolate Each Tile in the Reference Image:** The `all_tiles` board likely has a known arrangement (e.g. a 6×6 block of tiles, where each row is one shape and each column one color, or vice versa). We can exploit that structure: for example, if shapes vary by row and colors by column, we know tile at row1,col1 is Shape1-Color1, etc. If the arrangement is unknown, we can detect the tiles similarly to anchor detection:
- Compute differences from an empty board background (if provided in `imagini_auxiliare`) to find all occupied cells in the reference image.
- They should form 36 distinct cell regions. Cluster them by their grid positions (likely contiguous).
- **Extract Shape Contours:** For one representative tile of each shape type (we need six shape templates):
- Take a tile image patch (e.g. the first color of that shape row). Remove its background as we did for anchors (threshold out the board and tile background to get the symbol mask).
- Find the symbol's contour and **store it as the shape template**. We retain the contour points (or compute its Hu Moments).
- **Assign a Shape ID** to this template. If the assignment (1–6) isn't given explicitly, we define it systematically – e.g. the first shape encountered becomes ID1, next ID2, etc., or by the order in the plan PDF (ensure it matches the ground truth convention). Consistency is key: our shape numbering must align with the evaluation's expectation.
- Repeat for all 6 shapes. We now have `shape_templates = {1: contour1, 2: contour2, ... 6: contour6}`.

- **Extract Color Samples:** We need prototypes for the six colors (R, B, G, Y, O, W). Using the reference image:
- For each color, gather several sample pixels. One approach is to take the *mean HSV color* of each tile in a known color column, then average those means for all shapes of that color. For example, if column 1 has Red tiles of all shapes, compute each tile's average color and then average them to get a robust Red prototype.
- Alternatively, perform k-means clustering in HSV on all 36 tile color pixels into 6 clusters – each cluster center should correspond to a color. However, since we know the mapping from arrangement, a direct grouping is simpler.
- Store the color prototypes as HSV vectors: `color_proto = {R: (h1,s1,v1), B: (h2,s2,v2), ...}`. We will use these for nearest-neighbor matching.
- **Template Storage:** Save shape templates either as contours in memory or as binary images (for debugging). Save color prototypes as numeric arrays. (If performance is a concern, we could pre-compute the Hu Moments for each shape template now, to compare with new shapes quickly later.)
- **Template Verification:** It's wise to test the templates on known data:
- **Shape template check:** Run `cv2.matchShapes` between each pair of shape templates – it should give a very low score when matching a template to itself and high when to a different shape (except the square vs diamond case discussed below). This sanity check ensures contours were extracted correctly.
- **Color prototype check:** The six HSV prototypes should be well separated in hue space. Plotting them or printing them can confirm none are too close. If, say, Orange vs Red are somewhat similar, we note to be cautious (maybe Orange has higher H ~20° vs Red ~0°). We can also apply the classifier on the reference image itself to see if each tile is identified as the correct color and shape (all 36 should map correctly if templates are good).

By preparing these templates from a clean image, we ensure robust pattern matching during the game. We avoid using potentially noisy game frames to build templates (which could incorporate lighting or perspective artifacts).

Detecting New Tiles via Frame Differencing

Step 5: Identify newly placed tiles in each move by comparing consecutive frames. Starting from `j01.jpg` onward, for each move frame do:

- **Frame-to-Frame Differencing:** Compare the current frame with the previous frame (e.g. `j05` vs `j04`). Since only new tiles are added (no removals in Qwirkle), differences should highlight exactly those new tiles. We use a **cell-wise approach**:
- For each cell that was *empty* in the previous board state, compute an absolute difference between the current and previous frame in that cell region. This can be the sum of absolute pixel differences or another metric. We expect significant difference in cells where a new tile has appeared.
- Optionally, to reduce false positives from minor lighting changes, we can normalize the images (e.g. apply slight Gaussian blur or brightness normalization) or set a threshold high enough that only the strong signal of a tile triggers detection. We ignore cells that were previously occupied (anchors or already placed tiles) – even if lighting changes cause some difference, those are not *new* placements.
- **New Cell Detection:** Flag any cell with difference above a threshold as containing a new tile. This yields a set of candidate new positions for that move. Typically, Qwirkle moves range from 1 to 6 tiles. If our detection finds, say, 0 or 7+, we know something is wrong:

- If **no new cell is found**, it could be an error (every move frame should have at least one new tile). We would then lower the threshold or examine the global alignment. Possibly a very subtle tile (e.g. a dark tile in low light on a dark square) might not cross the threshold – in that case, adapt by looking at variance or edges in that cell.
- If **too many cells are flagged** (e.g. widespread changes), likely the image alignment is off due to camera shift. We then realign using known reference points (like anchor tiles positions) and recompute the differences.
- **Filter and Confirm New Tiles:** The initial difference mask might highlight *entire tile areas* plus some edges. We refine:
- For each flagged cell, consider the difference image – it should roughly match the shape of a tile. We can require that the region of difference covers a reasonable portion of the cell (not just a tiny spot). This filters noise (e.g. a small lighting change on an empty cell).
- If two adjacent cells are flagged, check if perhaps one tile was placed slightly off-center but not actually covering two cells:
    - If one cell's difference region is large and the adjacent's is very small or just on the border, likely it's one tile spanning the boundary. In that case, assign the tile to the cell where the majority lies, and discard the false neighbor detection. (We assume players place tiles roughly in one cell; a tile cannot count for two cells in a valid move.)
- **Identify Line Orientation:** Once we have the list of new tile coordinates for the move, we determine if they form a horizontal or vertical line:
- If multiple tiles, they must all share the same row or same column (or be contiguous with anchor gaps). Check their coordinates: if all new tiles have the same row, it's a horizontal placement; if same column, vertical. If neither, our detection likely picked up an error (since a valid move can't be scattered).
- If we find an inconsistency (e.g. two new tiles in different rows and columns), we revisit the difference images for false positives. It's possible a lighting artifact triggered a phantom tile. Drop the least convincing detection (the one with smallest difference magnitude) and re-check. The end result should be a single line of new tiles.
- **Incorporate Anchors in Line:** Determine if there are any gaps between these new tiles that are actually filled by existing tiles (anchors or previously placed). For example, if new tiles are at `10D`, `11D`, and `13D`, there is a gap at `12D` – if `12D` is an anchor, then the new tiles are forming a continuous line through that anchor. We note that for scoring but do not include the anchor in the "new" list (only newly placed tiles will be annotated in output). It does tell us the line is vertical in column D from 10 to 13.
- **Update Board State (tentative):** Before finalizing, we'll classify the new tiles in the next steps. But we can mark those coordinates as occupied in `board_state` now, to use for any subsequent move detection (so we know they are not empty next time).

This differencing approach is robust to moderate lighting changes and tile shifts. By focusing only on previously empty cells, we inherently ignore slight movements of existing tiles. For example, if an anchor tile jiggles between frames (causing difference), our logic sees that cell was occupied before, so it's not *new*. Thus, it won't mistakenly report it as a new placement. The combination of **state-tracking** (empty/occupied) and image differencing ensures reliable new tile detection.

Shape Recognition with Contour Matching

Step 6: Classify the shape of each new tile using contour matching. With the new tile cells identified, we now determine which of the six shapes has been placed:

- **Symbol Segmentation:** For each new tile cell (from Step 5), extract the symbol as we did for anchors:
- Crop the cell region from the current frame (with a slight padding if possible, to capture the full shape even if it touches the cell border).
- Generate a binary mask of the colored shape. We use the HSV approach: filter out the background by thresholding on low V (and maybe U channel if needed). The black tile background will have very low brightness, whereas the colored shape has higher brightness and saturation. For example, set a threshold like V > 50 and S > 100 (tunable) to isolate the symbol. This produces a clean mask where `1` = shape region, `0` = background.
- If the shape's color is very dark (like a deep blue or possibly purple), we adjust thresholds accordingly (lower V threshold or focus on S difference since black has S~0 and color has S high). We can also apply a slight blur before threshold to reduce noise.
- Use morphological operations if needed to close holes (though Qwirkle symbols are solid).
- **Contour Extraction:** Find the outline contour of the shape from the mask. We choose the largest contour (by area) which should correspond to the symbol's outer boundary. Simplify the contour (e.g. we can keep it as is or approximate it if extremely detailed – but we should maintain its distinctive geometry for matching).
- **Match to Templates:** Use OpenCV's `cv2.matchShapes` (which computes Hu moment based shape similarity) to compare the new tile's contour with each of the six template contours:
- This function gives a **match score** (lower means more similar) that is invariant to scale and rotation [3]. We compute score_i = matchShapes(new_contour, shape_template_i).
- Select the shape ID with the **lowest score** as the best match. In ideal conditions, the correct shape yields a score near 0, and others significantly higher.
- **Rotation Consideration (Square vs Diamond):** One caveat: Hu moments are rotation-invariant [4] [3]. This is usually desirable (we don't care how the shape is rotated in the image, we just want to recognize it), **but** Qwirkle has a "square" vs "diamond" which are the same shape rotated 45°. `matchShapes` will consider a square and a diamond identical (since one is just a rotated square). To distinguish these, we implement a special check:
- Compute the contour's **orientation**. For example, fit a bounding rectangle to the contour or use PCA to find the principal axis. If the shape's orientation angle is ~45° (± few degrees), and the top match is one of {square, diamond}, then classify it as the diamond; if it's ~0° or ~90°, classify as square. This works because in the board images the square shape is aligned with the grid, whereas the diamond shape is rotated on the tile.
- For other shapes (circle, star, clover, plus), rotation invariance is fine (they are symmetric or distinct enough).
- **Ambiguity Handling:** If the top two shape matches are very close in score (within a small delta), it signals uncertainty:
- For instance, plus "+" vs a clover might be vaguely similar if the image is noisy. In such a case, we can choose the one that leads to a **valid move** (using rule checks later). We postpone finalizing if ambiguous and mark it for validation.
- Generally, the template matching is reliable since shapes are quite distinct, except the known rotation issue above.

- **Assign Shape ID:** Assign the determined shape number to this tile. We now know e.g. “this new tile is Shape 4”.

Throughout this step, we ensure to handle **mask failures** gracefully. If a shape’s mask was broken (say part of the shape was not captured), the contour might be incomplete and `matchShapes` could give a weird result. To catch this, we check contour area: a full tile’s symbol has a known approximate size (from templates). If the extracted contour area is much smaller, we likely lost part of it (e.g. shape touching ROI edge and got cut). In that case, we can reprocess with a larger ROI or tweak threshold. We aim to feed a complete contour to the matcher for accuracy.

By using contour matching against templates, we leverage the fact that Qwirkle shapes have unique silhouettes. This method is **robust to scale and rotation** differences [3] – even if a tile is slightly closer to the camera or rotated, the Hu moments still match. It’s also robust to moderate image noise. We avoid naive template image correlation, which would fail under rotation or lighting changes; instead, structural shape descriptors make the classification reliable.

Color Classification with HSV Clustering

Step 7: Determine the color of each new tile using HSV color analysis. With the shape identified, we next classify one of six colors (Red, Blue, Green, Yellow, Orange, White/Purple (W)):

- **Isolate Color Pixels:** Using the same mask from shape segmentation (Step 6), we extract the actual color region of the tile:
- Apply the mask to the original image (bitwise-and) to get only the shape’s colored pixels. This removes any background and leaves just the colored symbol.
- Convert those pixels to HSV (if not already).
- **Compute Average Color:** Calculate the **mean HSV value** of the shape’s pixels. We could also take the median to reduce influence of highlights or shadows, but mean is usually fine given uniform colored symbols. Let `(h_new, s_new, v_new)` be this average.
- **Nearest-Neighbor in Color Space:** We compare this new tile’s HSV to our six prototype color values from Step 4. Using a simple distance metric (e.g. Euclidean or squared Euclidean in HSV space [5]):
- Compute `dist(color) = (h_new - h_proto)^2 + (s_new - s_proto)^2 + (v_new - v_proto)^2` for each candidate color.
- Choose the color with the minimum distance as the classification [5]. (We might weight the hue component more if needed, since hue distinguishes the colors strongly. Saturation/value are secondary but help separate e.g. red vs orange if their hues are close.)
- **Color Clustering Robustness:** Our prototypes come from the controlled lighting of the reference image. If the lighting in game images differs, the raw HSV might shift (e.g. all colors appear slightly darker). Because we take the nearest cluster, small shifts shouldn’t cause error – the correct color cluster should still be closest in HSV space. For added safety:
- We can allow a small tolerance: if the best and second-best color distances are very close, mark this classification as tentative and revisit in validation (especially if the second-best is a logically possible confusion, like Red vs Orange).
- If we had multiple samples per color (we do, from reference), we could use an SVM or simple threshold ranges for classification, but nearest neighbor is sufficient.
- **Assign Color Code:** Label the tile with the corresponding color letter (R, B, G, Y, O, W). For example, if the closest prototype was Orange, assign `O`.

- **Special Case – "W" Color:** The dataset uses `W` as one of the color codes. If this represents **White** (or possibly **Purple**, depending on how they labeled it), our color prototypes should include it. Assuming `W` is **White**: a white symbol on a black tile would be very high value, low saturation (distinct from all other colors). If `W` is actually Purple (some confusion in notation), then it will have a hue ~270° and moderate saturation. In either case, our clustering will handle it as just another distinct color cluster.

After this, each new tile now has a determined `(shape_id, color_id)` classification. For example, we might conclude a tile is `Shape 5` and `Color R` (red). We update the board state entry for that coordinate, e.g. `board_state[10,'D'] = (5,'R')`.

Double-check consistency: If multiple tiles were placed, quickly ensure they are not all classified as the exact same tile that already exists on the board (which would violate game rules of no duplicates in a line). Realistically, our shape/color templates won't misidentify so badly, but for instance if two new pieces in a color-line ended up with the same shape ID, that's a red flag. We then know at least one is wrong (since a line of same color cannot contain the same shape twice). We postpone final correction to the validation step next, where we systematically enforce rules.

So far, we have processed detection and classification for the new tiles in the move.

Scoring the Move According to Qwirkle Rules

Step 8: Compute the score for the move using Qwirkle scoring rules and bonus squares. Now that we know where the new tiles are and what they are, we integrate them into the game logic:

- **Incorporate New Tiles into Board State:** Mark each new tile's coordinate in `board_state` with its shape/color (if not already done). This ensures when counting lines we see a complete picture.
- **Identify the Main Line:** All tiles played in one move lie in one row or one column (plus possibly separated by existing tiles). Determine that direction:
- If multiple tiles share the same row, that row is the main line; sort their column positions. If multiple share same column, that column is the main line; sort by row.
- Include any **existing tiles (anchors or earlier moves)** that extend this line on either end. Starting from the smallest index to largest, move outward:
    - For a horizontal line: let minCol = min(new_tile_cols), maxCol = max(new_tile_cols). Check adjacent columns beyond the new tiles range within the same row: while `board_state[row, col_before]` is occupied (anchor/old), include it and decrement `col_before`. Do the same to the right (`col_after`).
    - For vertical: similarly include contiguous tiles above and below the new ones.
- Now we have the full continuous line of tiles that includes all new tiles and any pre-existing ones between or at the ends. Count the **line length** = number of tiles in this line.
- **Score the main line:** By Qwirkle rules, the score from a line is equal to the number of tiles in that line [6] [7]. (Each tile is 1 point essentially.)
- Check for **Qwirkle**: if line length == 6, that's a Qwirkle – add a bonus +6 points [8]. (The base line gives 6 points, plus 6 bonus, totaling 12 for that line [8].)
- **Score Perpendicular Lines:** Each newly placed tile could form **additional lines** in the perpendicular direction:
- For each new tile, look in the perpendicular direction (if main line is horizontal, check vertical through that tile; if main is vertical, check horizontal through that tile).

- Gather contiguous tiles in that perpendicular line (similar method: go up/down or left/right from the new tile until reaching an empty cell or board edge). Include any anchors or old tiles present.
- If the perpendicular line length is >= 2 (meaning the new tile connected with at least one existing tile in that direction), then this forms a valid line to score:
    - Count the tiles in this perpendicular line.
    - Add that count to the score.
    - If that line length is 6, add Qwirkle bonus +6 for that line as well.
- If the perpendicular line length is 1 (just the new tile alone with no neighbors in that direction), it doesn't score – essentially no line was formed there.
- **No double counting:** It's possible a new tile lies at the intersection of the main line and a perpendicular line – in scoring, **each line is scored independently** but each tile contributes in each line. (This means a new tile in two lines is counted twice, once in each line, which is correct for Qwirkle scoring [7].)
- **Add Bonus Square Points:** For each new tile played, check if its position is a bonus square:
- If `bonus_map[row,col] = 1` or `2`, add that many points to the score [9].
- **Important:** Only add once per tile even if it forms two lines [9] (our method naturally does this: we just add the bonus as a separate term per tile).
- For example, if two new tiles were placed on two separate bonus squares, and one is a +2 and the other +1, that adds +3 to the move score.
- **Calculate Total:** Sum the main line score, all perpendicular line scores, and all bonuses (Qwirkle + board bonuses). This is the total points earned in this move.
- **Examples:** To validate our scoring implementation, consider the examples from rules:
- A single tile placement that extended two lines (one of color, one of shape) and was on a 1-point bonus: If one line had length 2 and another length 2, base score 2+2=4 plus 1 bonus = 5 [7] – matches the example of the "green circle" scoring 5.
- Multiple tile placement example: two tiles placed forming a line of 3, and each tile also completed some perpendicular lines, plus both were on +2 spaces, gave 9 points [7]. Our algorithm would compute line lengths and bonuses accordingly (3 for main line + 2 + 2 for two perpendiculars + 2 bonus = 9).
- **Update Running Score (if needed):** If we were tracking cumulative score per player, we would add this to the player's total. But since the output needs only the move score, we just output this number for each move.

This scoring procedure strictly follows Qwirkle rules and accounts for all bonuses and special cases. We rely on the integrity of `board_state` to get correct line lengths. Because we explicitly traverse the board state for lines, off-by-one errors are avoided – we're counting actual tiles present, not just assuming counts. For instance, if anchors were in between new tiles, they are included so the line length is accurate.

Before finalizing, we do one more pass: **validate the move against rules** (see next section). This can also serve as a safety net to catch any scoring mistakes (if a move is invalid, its score is meaningless anyway). Once validated, we proceed to output.

Move Validation and Rule Enforcement

Step 9: Validate the new move using Qwirkle rules and correct any classification errors before output. We ensure the set of new tiles and resulting lines constitute a legal Qwirkle move:

- **All Tiles in One Line:** Verify all newly placed tiles are indeed in one row or one column (we determined this earlier). If by logic error we had two separate lines of new tiles, that's invalid. The pipeline already filters this, so at this stage, this condition should be met.
- **Shared Attribute Rule:** Tiles in a line must share one attribute (all same color or all same shape) and not have duplicates of the other attribute:
- Determine the line's attribute: Look at the new tiles (plus any anchors between them). If they all have the same **color**, it's a color-line. If they all have the same **shape**, it's a shape-line. If neither, the move is invalid.
- If supposed color-line, check that **no two shapes repeat** among all tiles in that line (including anchors). There are 6 unique shapes in the game; a valid line can have at most one of each. If we find a duplicate shape, something is wrong in classification.
- If supposed shape-line, check that **no two colors repeat** in that line.
- For example, if our classification resulted in a line with two tiles both identified as shape 4 (and it's meant to be a color line), likely one of those tiles' shape was mis-read.
- **Touching Existing Tiles:** Check that at least one of the new tiles is adjacent to an existing tile (anchors or prior placed) – except possibly the very first move of the game (which in Connect isn't free-floating anyway, it had to touch anchors). In practice, by difference detection we inherently assume they attach, but it's a rule: no isolated placements. Our scoring logic would have found 0 perpendicular line for all new tiles if they were isolated, which would violate "must touch already on board" rule. So if a move yields *no* perpendicular connections and it's not the first move, it may be illegal (or just extending an existing line from one end, which is fine actually – touching happens along the main line in that case).
- **No Gaps:** Ensure continuity: between the min and max of the new tile positions, every intervening cell in that row/col is occupied (either by a new tile or an existing one). If we detect an empty gap in between, the move is invalid (players can't skip spaces). Our line assembly logic already accounts for this by including anchors; if a gap remained, it means a tile is missing. That would indicate a detection failure (maybe a very similarly colored anchor wasn't detected initially). If that happens, we might need to revisit anchor detection or consider that the supposed gap might actually have a faint tile that was missed.
- **Reclassify on Rule Violation:** If any rule is violated, we attempt to fix classification:
- Identify which tile is suspect. E.g., if in a color-line we have a duplicate shape, one of those two identical shape IDs is wrong. We look at their matchShapes scores – perhaps the second-best match for one of them was a different shape that would resolve the conflict. We then recompute that tile's identity with the alternate shape. Essentially, choose the classification that makes the line valid (since we know the move made by a human must be valid).
- Similarly, if colors have a conflict in a shape-line (duplicate color), swap the tile's color to the second-closest prototype if that yields unique colors.
- After an adjustment, re-run scoring for that move (the shape/color change might affect perpendicular lines too). Usually, only one tile will be misclassified in a way that violates rules, so a single correction suffices.
- Because the game's rules are strict, this is a powerful sanity check. For example, if our image analysis mis-read a dark orange as red, we'd have two "red" tiles in what should be a shape-line of different

colors – rules flag it, and we can realize that tile was likely orange (the next closest color) which would fix it.
- **Final Board State Update:** Once validated, the new tiles are confirmed in `board_state`. We lock in their shape/color. This state now accurately represents the board after the move.

By enforcing these checks, we **catch errors early**. It is extremely unlikely for a move to pass all validations if any tile was misidentified. The redundancy between computer vision and rule logic makes the solution robust: even if lighting or segmentation caused a mistake, the inconsistency with game rules will reveal it. Our pipeline then either corrects it or, in worst-case, could flag an error (though for a homework solution, automatic correction is expected).

Now the move is ready to be reported.

Output Formatting of the Results

Step 10: Format the detection and scoring result for the move in the required .txt format. We output one text file per move image, as specified:

- **List Newly Placed Tiles:** For each tile placed in the move, output a line with the **board coordinate** and the **tile code**. The tile code is composed of the shape number (1–6) and color letter (R,B,G,Y,O,W) with no space between them. Format: `<Row><Col> <Shape><Color>`. For example: `15D 20` means row 15, column D, shape 2, color Orange.
- **Order of Listing:** We output one line per new tile, then a final line for the score. The ordering of the tile lines should be consistent and logical:
- By convention, we sort by row, then column (which also means left-to-right for horizontal lines or top-to-bottom for vertical). This matches the sample outputs. For instance, if tiles were at 15C, 15D, 15E, 15F, we list in ascending column: 15C, 15D, 15E, 15F [10]. If tiles were at 10D, 11D, 13D (vertical, skipping 12D anchor), those should appear sorted by row: 10D, 11D, 13D [11].
- This ordering also naturally groups them in the line sequence they appear on the board.
- **Score Line:** After listing all tiles, output a final line with just the **integer score** for that move (no label or extra text). For example: `8` on the last line if 8 points scored [10].
- **No Extra Spaces or Lines:** Ensure each line has the format exactly as expected: coordinate and tile separated by a single space, and the score as a single number. No trailing spaces. Use the same capitalization as defined (columns uppercase A–P, color letters uppercase).
- **Example output for a move:** If the move placed 3 tiles at 2D (shape5 red), 2E (shape1 red), 2F (shape3 red) and scored 7, the file should contain:

```
2D 5R
2E 1R
2F 3R
7
```

  This matches the ground-truth format [12].
- **Filename convention:** Though not part of text content, ensure to save the file with the correct naming (likely `<game>_<move>.txt`, e.g., `1_05.txt` for game1 move5).

We should also double-check that the shape and color IDs we output correspond **exactly** to the ground truth's definition. If there was any ambiguity (like what "W" stands for), it should be resolved by comparing with provided examples. E.g., if ground truth shows a tile with color W in an example and we identified it as Purple, then indeed W must correspond to that purple tile. We must use `W` in output to match, regardless of naming.

Finally, we verify our output by running the provided evaluation script on the fake_test images if possible, to ensure formatting and values are perfect. This catches any minor mistakes like ordering or off-by-one in scoring. Once the output format is confirmed correct for sample moves, we proceed to full execution on all test images.

Debugging and Visualization Techniques

Throughout development, we employ several **debug and visualization aids** to ensure each pipeline step is working and to identify where errors occur:

- **Visual Difference Highlights:** Create images showing the difference between frames with new tiles circled. For example, after Step 5 (detection), draw a red outline on cells where a new tile was detected. This helps verify that our differencing is picking the correct cells and not missing any or flagging falses.
- **Contour Overlay:** During shape classification (Step 6), overlay the extracted contour on the tile image in a debug window or saved image. We can also draw the best-match template contour in a different color for comparison. This visually confirms that the contour extraction was correct and that the match orientation is sensible.
- **Color Mask Display:** Show the masked colored region for each tile and print the computed HSV vs prototype HSV values. This can reveal if a lighting change is shifting colors significantly (e.g. all reds appear darker – if so, we might adjust our threshold or consider normalizing brightness).
- **Logging and Assertions:** Print out intermediate results in text logs:
- List of detected new tiles each move (coordinates).
- The shape match scores for each template (to see if any are close).
- The chosen shape and color for each tile.
- The computed line lengths and score breakdown (e.g. "Main line length 4 + perpendicular 2 + Qwirkle + bonus = total" for transparency).
- Any rule violation messages encountered and how they were fixed.
- **Step-by-step verification:** Use the provided ground-truth for fake_test to compare at each stage:
- After classifying anchors (initial board), cross-check a few against the ground truth of move 0 if available (or known initial setup). This ensures templates are correct.
- After each move, verify that the set of new tiles and score match the ground truth (during development). The evaluation script is handy for this – run it in verbose mode for each move to see if position, tile, score all match exactly [13] [14] . If something differs, use the debug outputs to pinpoint which step made a mistake.
- **Visualization Tools:** We can use OpenCV's GUI windows (cv2.imshow) for quick visual checks if running locally, or save annotated images to disk. For instance, draw the entire board state after each move with different colors or labels on tiles to track progression.
- **Exception Handling:** Incorporate checks in code that assert expected conditions – e.g., "if len(new_tiles)==0: raise Exception('No new tile detected')" so we don't silently produce wrong output. These should be removed or handled gracefully in the final version, but are useful during testing.

Using these techniques, we can iteratively refine the pipeline. By the time we finish debugging, each component from calibration to output will have been validated, and the entire pipeline can run through all moves automatically, with high confidence in its robustness.

Modular Code Architecture

To manage complexity, we organize the solution into clear modules/classes, each handling a specific responsibility. This separation also aids in debugging and testing individual parts. A proposed architecture:

- **BoardCalibrator**: Handles all board-level calibration.
- Methods: `find_board_roi(empty_image, frame0)` to get homography/ROI, `warp_image(image)` to apply perspective transform, `board_to_image(coord)` and `image_to_board(pixel)` for mappings.
- Also includes `determine_orientation(anchor_positions)` to set up the bonus mapping.
- Stores: `homography`, `cell_size`, `bonus_map` (maybe).
- **TemplateExtractor**: Responsible for loading the auxiliary images and extracting shape and color templates.
- Methods: `extract_shape_templates(all_tiles_image)`, `extract_color_prototypes(all_tiles_image)`.
- Returns or stores: `shape_templates` (contours or Hu moments), `color_prototypes` (HSV means).
- Could also have helper to handle the square/diamond orientation logic (store which template is square vs diamond).
- **BoardState**: A structure (could be a class or just a dict in code) that keeps track of the current board.
- Methods: `place_tile(coord, shape, color)` updates state, `remove_tile(coord)` (not used in Qwirkle as no removals), `is_occupied(coord)` check.
- Method: `get_line(coord, direction)` to get all tiles in a line (for scoring).
- The state can be a 16x16 array or dict mapping (row,col) -> (shape,color).
- **TileDetector**: Handles new tile detection between frames.
- Method: `find_new_tiles(prev_image, curr_image, prev_state)` -> returns list of coords for new tiles.
- Internally uses differencing as described. Likely needs `BoardCalibrator` to align/warp images first, and uses `prev_state` to ignore occupied cells.
- Could also produce a diff image for debug.
- **TileClassifier**: Handles shape and color classification for a given tile region.
- Methods: `classify_tile(image, coord)` -> returns shape_id, color_code.
- Internally: does segmentation mask, contour extraction, shape match, color analysis, orientation fix for diamond, etc.
- Uses `TemplateExtractor`'s data (shape_templates & color_prototypes).
- Optionally returns the match scores or a confidence measure for validation use.
- **MoveValidator**: (Could be part of BoardState or separate) Implements the rule checks.
- Method: `validate_move(new_tiles, board_state)` -> returns True/False and possibly corrected classifications.
- Checks contiguous line, shared attribute, no duplicates, etc. If corrections are needed, it might call back into `TileClassifier` with alternate options (or simply adjust the results).
- Ensures that after this, `board_state` can be updated with confidence.

- **ScoreCalculator**: Computes the move score given `board_state` and the list of new tile coords.
- Method: `compute_score(new_tiles, board_state, bonus_map)` -> returns int score.
- Implements the line counting and bonus addition logic described.
- **SolutionWriter**: Handles formatting and writing output files.
- Method: `write_move(filename, new_tiles, board_state, score)` – writes the lines in the correct format. It can sort the new_tiles list as required and map numeric IDs/letters to strings.
- Could also zero-pad move numbers in filename if needed and manage directories.
- **MainPipeline** (or orchestrator script): Puts it all together.
- Loads images in sequence for each game.
- Calls `BoardCalibrator` on the first game's first frame (with empty board image).
- Calls `TemplateExtractor` once to get templates.
- Uses `BoardState` to store anchors after initial detection.
- Loop through moves 1..N:
    1. TileDetector to get new tile positions.
    2. For each position, call TileClassifier to get shape/color.
    3. MoveValidator to ensure the set is consistent (possibly adjust if needed).
    4. ScoreCalculator to compute the move's score.
    5. SolutionWriter to output the .txt file for that move.
    6. Update BoardState with the new tiles.
- Repeat for each move, and similarly for each game if multiple games are evaluated.

Each component can be unit-tested or run in isolation with known inputs (e.g. test TemplateExtractor on the aux image and verify it found 6 shapes, 6 colors correctly; test TileDetector on a pair of frames with a known new tile). This design makes the system **extensible and maintainable**. If, for example, the method of shape recognition needed to change (say to a CNN classifier), we could replace the TileClassifier implementation without touching detection or scoring logic.

By dividing tasks, we also handle errors at the appropriate level: e.g., if TileDetector fails to find a tile, BoardState/MoveValidator will catch an illegal move (no new tile) and we know to adjust detection. If classification is ambiguous, MoveValidator catches it and could query TileClassifier for alternatives.

Pipeline Execution Flow

Finally, here is a step-by-step **execution flow** tying all components together (in pseudo-code style):

```
# Pre-load resources
empty_board = cv2.imread("imagini_auxiliare/empty_board.jpg")
all_tiles_img = cv2.imread("imagini_auxiliare/all_tiles.jpg")

# Initialize main components
board_calib = BoardCalibrator(empty_board)
template_ext = TemplateExtractor()
shape_templates, color_prototypes =
template_ext.extract_templates(all_tiles_img)
tile_detector = TileDetector(board_calib)
tile_classifier = TileClassifier(shape_templates, color_prototypes)
```

```
move_validator = MoveValidator()
score_calc = ScoreCalculator(board_calib.get_bonus_map())
board_state = BoardState()

# Initial frame j00 processing (anchors)
frame0 = cv2.imread("game1/j00.jpg")
frame0_warped = board_calib.warp_image(frame0)
anchor_positions = tile_detector.detect_anchors(frame0_warped,
empty_board_warped=None)
for pos in anchor_positions:
    shape_id, color_id = tile_classifier.classify_tile(frame0_warped, pos)
    board_state.place_tile(pos, shape_id, color_id)
# Determine orientation and bonus mapping
board_calib.determine_orientation(anchor_positions)
score_calc.update_bonus_map(board_calib.get_bonus_map())
# (Reclassify anchors if needed after orientation - probably not needed unless
we want to label which are 1-point vs 2-point, but anchors' shape/color are
done)

# Process moves j01 ... j20
prev_frame = frame0_warped
for move in range(1, total_moves+1):
    curr_frame = cv2.imread(f"game1/j{move:02d}.jpg")
    curr_frame_warped = board_calib.warp_image(curr_frame)

    new_tiles = tile_detector.find_new_tiles(prev_frame, curr_frame_warped,
board_state)
    new_tiles_info = []  # will hold (pos, shape, color)
    for pos in new_tiles:
        shape_id, color_id = tile_classifier.classify_tile(curr_frame_warped,
pos)
        new_tiles_info.append((pos, shape_id, color_id))
    # Validate and correct if needed
    move_validator.validate_move(new_tiles_info, board_state)
    # Now update board state with corrected info
    for pos, shape_id, color_id in new_tiles_info:
        board_state.place_tile(pos, shape_id, color_id)
    # Calculate score
    move_score = score_calc.compute_score(new_tiles_info, board_state)
    # Write output
    SolutionWriter.write_move(f"solution/game1_{move:02d}.txt", new_tiles_info,
move_score)
    # Set up for next iteration
    prev_frame = curr_frame_warped
```

(This is a simplified sketch; actual implementation will handle indexing, multiple games, and integrate debugging as needed.)

Each iteration processes one move, detecting new tiles, classifying them, validating the move, scoring it, and outputting the result. The `prev_frame` is updated to the current frame to detect the next move's difference.

With this structured approach, the system ensures **safety and reliability** at each step, using fallbacks and checks to handle real-world imperfections (camera misalignment, lighting, tile shifts, etc.).

Robustness: Handling Potential Failure Modes

Even with the above careful design, we anticipate certain failure modes. Here we list major ones, explain why they might occur, how to detect them early, and the mitigation strategies integrated in our plan:

- **Tile Misalignment Across Cells:** *Tiles might not be perfectly centered in a cell (e.g. slid towards a neighbor cell).* This can cause the differencing step to flag two adjacent cells or the shape mask to be cut at the border.
- **Detection:** We check if two neighboring cells are both detected as "new" but one has only a small fraction of the tile. Also, the shape's contour might extend to the very edge of the ROI, indicating it wasn't fully captured.
- **Mitigation:** We assign the tile to the cell where the majority of its area lies (based on difference intensity or contour centroid). We also slightly **expand the ROI** when extracting the shape mask, so if the symbol touches the border, we still capture it fully. Our validation step will also catch impossible scenarios (like two "new" tiles in adjacent cells that would violate the one-line rule). The pipeline then knows it must be one tile mis-read and correct it by merging those detections.
- **Lighting Changes Between Frames:** *Global illumination may vary (camera auto-exposure, time of day sunlight changes), causing many cells to show differences or colors to shift.*
- **Detection:** We might see a high baseline difference across the board (many cells slightly above threshold). Also, color classification distances might consistently be larger than expected (none close to prototypes, because everything is darker/lighter).
- **Mitigation:** We set a robust threshold for detecting new tiles that ignores small intensity changes – essentially looking for *significant* change. We can subtract a blurred version of the difference image to filter out global lighting variation. If needed, we could implement a frame-to-frame brightness normalization: e.g. compare histograms of the previous and current frame and adjust the current frame's brightness accordingly before differencing. For color shifts, we could update our `color_prototypes` by recalibrating on the anchor tiles: since we know each anchor's color from initial classification, we can measure what HSV values they have in the current frame and slightly adjust our prototype means if there's a consistent offset. This dynamic calibration ensures color classification stays accurate despite lighting changes.
- **Shape Mask Extraction Failures:** *The colored symbol might not be cleanly segmented from the background.* This could happen if the color is very dark or if part of the shape overlaps a similarly colored background element.
- **Detection:** The extracted contour area is much smaller than expected (e.g. a tiny contour when we know the tile symbol should occupy most of the tile). Or multiple disjoint contours appear (the shape got broken into pieces).
- **Mitigation:** We adjust thresholding: e.g. for a dark blue symbol on black, increase the sensitivity (lower the V threshold). We can also incorporate edge-based detection: run Canny edge, then close the edges to form a contour. If the fill is not coming through due to low contrast, the edges might still delineate the shape. We then fill the closed contour to recover the full symbol area. Our pipeline

could try both methods – if pure HSV masking yields too small a contour, switch to edge detection for that tile. We also ensure to capture a bit of the tile border in the ROI; even if the symbol touches the tile's edge, we might see a contrast between the symbol and the board background (green), aiding edge detection at that boundary.

- **Weak Shape Template Matches (Ambiguity):** *Some shapes might produce similar match scores.* As noted, square vs diamond will confuse a rotation-invariant matcher.
- **Detection:** Check the ratio of the best match score to the second-best. If it's close (say >0.9), the identification is ambiguous. Also, if we know certain pairs are problematic (square/diamond), we explicitly watch for that scenario (best match is square and second best is diamond with almost the same score).
- **Mitigation:** We incorporate the **orientation heuristic** for square/diamond: measure the contour's angle to decide which it is. For other potential ambiguities, use contextual clues – e.g., if a tile is supposed to form a color line and we already have certain shapes present, the new one cannot be one of those (if our classifier picks an already present shape, that's likely wrong due to the no-duplicate rule). We then consider the next best shape that isn't present. This uses game logic to break ties. Additionally, we could augment matchShapes with a second method: e.g. Hu moments are one metric; we could also compare normalized Fourier descriptors of the contour. Agreement between methods gives confidence; disagreement triggers a closer look.
- **Classification Mismatch Violating Rules:** *A tile's shape or color is misclassified such that the move as a whole breaks a Qwirkle rule.* For example, identifying two tiles as the same shape in a color-line.
- **Detection:** The **MoveValidator** explicitly checks for attribute consistency and duplicate shapes/colors. Any violation here is a direct indicator of a classification error.
- **Mitigation:** We programmed the validator to attempt automatic correction. It will try alternate classifications for the offending tile(s):
    - If two new tiles share shape but should be a color-line, determine which one is likely wrong by looking at match scores or color similarity. Perhaps one shape's second-best match is a different shape that would make all shapes unique – switch to that.
    - If a color seems off (e.g. a tile read as Red but all others are Orange), check the color distance: maybe it was borderline and the second nearest is Orange. Switch it.
    - After a correction, re-run the validation to ensure everything is consistent. This feedback loop continues until the move is valid or no alternative yields validity (in which case something more fundamental is wrong, though that should not happen with correct game data).
- This strategy effectively uses the **game's constraints as a guide** to fix CV mistakes, greatly enhancing reliability.
- **Incorrect Score Calculation (Off-by-One or More):** *The computed score might be wrong if we mis-count tiles in lines or mis-apply bonuses.* This could happen if we didn't include an anchor in the line count or double-counted a bonus.
- **Detection:** During testing, we compare our output against ground truth scores. An off-by-one score is easy to spot. Also, logically, certain moves have expected patterns (e.g. placing one tile that doesn't complete a Qwirkle and not on a bonus must score at least 1 and at most 1 + some perpendicular lines – if we got 0 or some too-large value, it's clearly wrong).
- **Mitigation:** Our scoring uses explicit board state traversal:
    - By traversing until an empty cell, we ensure **no missing or extra tiles** in the count. If a tile was missing in board_state (due to a detection error), the line would cut off early – but that scenario would likely be caught earlier by validation.
    - We double-check adding of bonuses: each new tile's bonus is added exactly once. The code structure makes it unlikely to double-add, but we remain cautious with how we accumulate score.

    - We test edge cases: Qwirkle scenarios (6 in a line) to ensure +6 is added, multiple Qwirkles at once (rare, but if a single placement completes two lines of 6, we should add +6 twice, once per line). We simulate such a case in a test to verify.
    - If any discrepancy is found, we inspect the breakdown (which we can log: e.g. "line scores = X, bonuses = Y") to pinpoint the error.
- **Board ROI Cropping Error:** *If the board ROI is off by a few pixels, some tiles at the edges might be partially out of frame or mis-located in the wrong cell.*
- **Detection:** Anchors at the edges in `j00` might have been segmented oddly (half missing). Or the empty board alignment shows a border of the board cut off. Numerically, if the transformed board image has black borders or misaligned grid lines (we can overlay a grid and see if lines match tile boundaries).
- **Mitigation:** Recompute the homography with higher precision:
    - Use subpixel corner detection for the board if possible (if the board has a distinct border or markings).
    - Perhaps detect the 16×16 grid lines across the board: using Hough line detection to find all vertical and horizontal grid lines. Their intersections give the grid corners which can refine the mapping.
    - We also included a margin in cropping to ensure even if alignment is slightly off, no tile is cut. We might capture a bit of the surrounding area and then mask to just inside the board.
    - If a consistent offset is noticed (e.g. every tile appears 2px lower than center), we incorporate that offset into subsequent calculations or adjust the homography accordingly.
- **Bonus Square Mis-mapping (Wrong +1/+2 assignment):** *If quadrant orientation was wrong, a tile on what we think is +2 might actually be +1, or vice versa, skewing the score by 1 point.*
- **Detection:** This would surface as a **score mismatch** when evaluating against ground truth – specifically a difference of exactly 1 (or 1 per such tile). If multiple moves consistently have a score off by these small amounts, suspect bonus mapping.

- **Mitigation:** If we have access to one fully correct example (like fake_test ground truth), we can deduce orientation from that (but let's assume we do it autonomously). Our orientation detection uses all anchors to avoid error, but if a quadrant pattern were symmetric in a way that fooled our matching, we might assign some 2's as 1's.

    - To fix, we could use an **observational approach**: consider the first move where a new tile lands on any black bonus square. The ground truth score (if known during development) would tell us if that was a +1 or +2 (score difference beyond tile count). We could then adjust our mapping accordingly. In automated evaluation without ground truth, this is tricky – so we rely on correctly matching the quadrant pattern initially.
    - We can also double-check the count of each bonus type: by rule, how many 2-point spaces exist (likely 8 or 12)? If our orientation assignment yields an implausible distribution (say 5 of one type 7 of another, when it should be equal), that hints a rotation error.
    - Ultimately, careful pattern matching with the known board diagram prevents this. We will test our `determine_orientation` on the auxiliary recommended setup image (if available) to ensure it labels bonuses correctly before applying to actual games.

- **Tile Movement or Removal (Unexpected):** *Although game rules don't remove tiles, a physical bump might move an anchor or dislodge it.* If an anchor slides significantly between frames, our algorithm might see its original cell as "lost tile" and new cell as "new tile".

- **Detection:** We would detect a previously occupied cell becoming empty and a neighboring empty becoming occupied – a swap. The evaluation script would likely mark our output wrong if we output a "new tile" where it was actually just moved, since ground truth wouldn't list a new tile. Also, a removal is not allowed, so any empty where a tile was is suspicious.
- **Mitigation:** We can encode that *no tile should ever disappear*. If our diff detection ever flags an occupied cell as empty, we ignore that (treat it as noise). If it flags a new tile in a cell adjacent to a now-empty former tile, interpret it as the tile moved. In such a rare event, one strategy is to assume the tile stayed in its original coordinate (i.e. ignore the movement, keep state as is). Another is to update the tile's coordinate in board_state (but that contradicts rules; likely better to stick to logical rules and assume the camera saw it moved but logically it's same tile).
- For a homework project context, we expect ideal or near-ideal conditions, so this may not occur. But it's worth noting as a potential real-world issue.

Each of these failure modes is addressed by our pipeline either inherently (through the design) or via explicit checks and fallbacks. The overall philosophy is: **detect anomalies early** (e.g. through validation or obvious inconsistencies) and then **either automatically correct them or adjust parameters** to handle them. By combining computer vision with game rule logic and multiple detection methods (color + shape + difference + orientation), the solution is very robust against noise and unforeseen issues.

In summary, this plan provides a **comprehensive, step-by-step approach** to solve the Qwirkle Connect score calculator task. It balances classical CV techniques (differencing, contour matching, color clustering) with domain-specific knowledge (Qwirkle rules, board layout) to achieve a high-reliability solution. Each step has been carefully considered to minimize errors, and contingencies are in place for the most likely failure scenarios. With thorough testing and validation, this pipeline will accurately detect every move and compute scores exactly as required, producing correct annotations for every image frame of the game. [3] [5]

---

[1] [2] [7] Qwirkle_BigBox_Rules_121115
https://www.gokids.com.tw/tsaiss/MindWare/rules/QwirkleBigBoxRules.pdf

[3] c++ - Rotation and scale invariant template matching in OpenCV - Stack Overflow
https://stackoverflow.com/questions/12601583/rotation-and-scale-invariant-template-matching-in-opencv

[4] Shape Matching using Hu Moments (C++ / Python) | LearnOpenCV
https://learnopencv.com/shape-matching-using-hu-moments-c-python/

[5] add_pieces_mosaic_all.py
file://file_000000008354720a952675b5355ff9e0

[6] [9] How to play Qwirkle Connect | Official Rules | UltraBoardGames
https://www.ultraboardgames.com/qwirkle/qwirkle-connect.php

[8] [PDF] Download Instructions - MindWare
https://www.mindware.orientaltrading.com/pdf/instructions/52132.pdf

[10] 1_03.txt
file://file_000000001024720a9a53f23fff585979

[11] 1_01.txt
file://file_00000000fd3c720a99fbb8d7701e9a58

12 1_02.txt
file://file_00000000237c71f499bab206a8532e9f

13 14 evalueaza_solutie.py
file://file_00000000e05471f487a74f63c7b79eaf